

LAB REPORT: REACTIVE NAVIGATION USING TURTLEBOT3 LIDAR

1. Introduction & Objectives

This report documents the successful implementation and completion of Lab Manual 6 for the MCT-454L Mobile Robotics course. The primary objective of this laboratory exercise was to utilize LaserScan data from a 2D LiDAR sensor mounted on a TurtleBot3 (Burger) to generate real-time motion commands for reactive navigation within a simulated Gazebo environment.

The core competencies achieved include:

- Interpreting and subscribing to the `/scan` topic data structure in ROS 2.
- Filtering data arrays to extract critical directional distances (Front, Left, Right regions).
- Designing algorithmic logic to map raw physical sensor thresholds directly to directional velocities.
- Implementing sub-behaviors including Obstacle Avoidance, Stop-on-Obstacle, and Wall Following.

2. System Setup & Data Understanding

The pre-lab environment was initialized using ROS 2 by configuring the TurtleBot3 Burger model environment variables and launching the `turtlebot3_world.launch.py` script inside Gazebo. The active nodes and topics were verified via `ros2 topic list`, ensuring that the `/scan` and `/cmd_vel` channels were actively operating.

The `sensor_msgs/msg/LaserScan` message structure was evaluated to parse out specific fields:

- `ranges[]`: Float array indicating obstacle distances in meters.
- `angle_increment`: Radians between consecutive readings used to map indices to degrees.

3. Algorithmic Implementation Details

A custom ROS 2 node, `LidarNavigator`, was completed based on the boilerplate guidelines. The algorithm segments the 360-degree array into three definitive zones to handle reactive behaviors:

- Data Cleaning:** Out-of-bounds or erroneous entries containing `inf` or `NaN` values were filtered and replaced with a predefined maximum sensor range using NumPy utilities to prevent navigation errors.
- Region Allocation:** The array was divided into three regions: Front (-15° to $+15^\circ$), Left (15° to 90°), and Right (270° to 345°).
- Control Decisions:** Proportional control was introduced to maintain linear speed adjustments. When a physical obstacle breached the `front_threshold` (0.5 meters), linear movement ceased, and the node compared the minimum distance metrics of the left and right sectors to switch angular rotation toward the path of maximum clearance.

4. Behavioral Testing & Observations

During live simulation runs, the individual navigational milestones met the designated requirements:

Behavior Task	Observed Performance Metric	Status
Stop-on-Obstacle	Robot successfully brought linear velocity to 0 m/s when approaching a direct obstacle.	Passed
Obstacle Avoidance	Evaluated side clearance and successfully steered into non-blocked paths.	Passed
Wall Following	Maintained a stable distance envelope from walls using proportional error scaling.	Passed

Observations on Instability and Tuning: Initial testing showed minor oscillations (fishtailing) when the robot ran parallel to a wall. This was corrected by reducing the maximum angular speed limits to 0.4 rad/s and applying an averaging filter to smooth out sudden sensor noise spikes.

5. Deliverables & Conclusion

All mandatory deliverables outlined by the lab manual instructions have been generated, packaged, and verified:

- **Source Code & Package:** The fully written ROS 2 navigation code has been committed and pushed to the designated GitHub repository.
- **Visual Verification:** High-resolution screenshots of the `rqt_graph` layout proving proper active subscription to `/scan` and publishing actions to `/cmd_vel` have been documented alongside RViz visualization frames.

In conclusion, this lab successfully provided hands-on experience in basic reactive navigation loop design. Processing raw sensor streams to command physical actuators in real time highlights the essential foundations of autonomous robot path planning and collision management systems.